Chapter 3 · Solutions

P09 · Rows versus customers

Hint. One purchase produces one joined match.

The joined names are Asha,Asha,Ravi: 3 rows. DISTINCT returns Asha,Ravi: 2 rows. EXISTS also returns 2 customer rows because each customer has a purchase.

Adding (1,40) gives Asha,Asha,Asha,Ravi: 4 joined rows. DISTINCT and EXISTS still return 2. EXISTS tests whether a witness exists; it does not output one row per witness.

P10 · Missing values and empty input

Hint. List the non-null inputs before calculating the aggregate.

Quantity	All rows	WHERE Value>100
COUNT(*)	4	0
COUNT(Value)	3	0
COUNT(DISTINCT Value)	2	0
SUM(Value)	40	NULL
AVG(Value)	40/3	NULL

The ungrouped aggregate returns one result row even on empty input. GROUP BY Value on that input returns zero groups and zero rows.

Check. NULL is ignored by the value aggregates; it is not replaced by zero. SUM(empty) is not the same SQL result as summing observed zero values.

P11 · Filter then group

Hint. Apply amount>8 before making groups.

Initially WHERE leaves (A,10),(A,20). Their A group sums to 30 and passes HAVING. The output is (A,30).

After adding (B,30), WHERE leaves those two A rows and (B,30). Both groups sum to 30 and pass. The output is (A,30),(B,30). B's original 7 and NULL are excluded before grouping.

Before WHERE, B has COUNT(*)=2 and COUNT(amount)=1. After the added row those become 3 and 2. They disagree because one row has NULL amount.

P12 · Correlated comparisons

Hint. Count strict inequalities. Do not assign a shortcut rank to tied rows.

Balance	Above	Below	Keep?
5, first row	3	0	yes
5, second row	3	0	yes
10	2	2	yes
15	1	3	no
18	0	4	no

The count is 3. After replacing 15 by 10, each 10 row has 1 strictly above and 2 strictly below, so neither passes. Each 5 row still passes. The count becomes 2.

P13 · NOT IN and NOT EXISTS

Hint. Evaluate an unmatched ID against every Rejected value.

For IDs 1 and 3, IN returns UNKNOWN because of the NULL member and no true equality. NOT IN remains UNKNOWN, so WHERE rejects them. ID 2 is explicitly present and is also rejected by NOT IN. Output: empty.

NOT EXISTS finds a real equality match only for ID 2. It returns 1 and 3. After removing NULL, both queries return 1 and 3.

5>ANY(empty) is FALSE. 5>ALL(empty) is TRUE: there is no element that violates the comparison. This empty-set convention follows the quantifiers, not a MAX(empty) substitution.

P14 · Preserving rows and ordered pairs

Hint. ON forms the join. WHERE can then remove its preserved rows.

With the amount condition in ON, the rows are (Asha,20),(Ravi,NULL). With it in WHERE, only (Asha,20) remains. NULL>15 is UNKNOWN, so Ravi does not pass.

For self-join values 1,1,2, each of the two value-1 rows matches the one value-2 row under <. There are 2 ordered pairs. Under ≤, the two value-1 rows match both value-1 rows and the value-2 row: 2×3=6 pairs. The value-2 row matches itself: 1 more. Total: 7.

Check. A self join includes the same underlying row on both sides unless the predicate rules it out.